

Математическое моделирование

Лабораторная работа № 2

Наталья Ларина

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение лабораторной работы	8
3.1	Нахождение радиуса перехода x	8
3.2	Разложение скорости катера и система ОДУ	9
3.3	Условие задачи	10
4	Полярные траектории: катер (ODE) и лодка (аналитически)	12
4.1	Инициализация проекта и загрузка пакетов	12
4.2	Параметры задачи	13
4.3	Определение модели	13
4.4	Запуск 1: $r_0 = s/(n+1)$, $t \in (1e-9, 8)$	15
4.5	Запуск 2: $r_0 = s/(n-1)$, $t \in (1e-9, 15)$	15
5	Параметрическое исследование: катер (ODE) и лодка (аналитически) в полярных координатах	17
5.1	Активация проекта и загрузка пакетов	17
5.2	Определение модели	18
5.3	Определение параметров в Dict	18
5.4	Функция-обертка для запуска одного эксперимента	19
5.5	Запуск базового эксперимента (кэширование)	21
5.6	Визуализация базового эксперимента	22
5.7	Второй базовый эксперимент (как у тебя во 2-й части): case=:minus, tmax=15	23
5.8	Параметрическое сканирование	24
5.9	Запуск всех экспериментов и сбор результатов	25
5.10	Анализ и визуализация результатов сканирования	27
5.11	Бенчмаркинг с разными параметрами	29
5.12	Сохранение всех результатов	31
5.13	Анализ результатов моделирования	32
5.14	Базовые эксперименты	33
5.15	Параметрическое сканирование по n	35
5.16	Анализ метрики scale_ratio	36
5.17	Время вычислений	37

6 Выводы	38
Список литературы	39

Список иллюстраций

5.1	Базовый эксперимент (case=plus)	33
5.2	Базовый эксперимент (case=minus)	34
5.3	Сканирование траекторий катера	35
5.4	Зависимость scale_ratio от n	36
5.5	Зависимость времени вычисления от n	37

Список таблиц

1 Цель работы

Рассмотрим пример построения математической модели, которая помогает выбрать рациональную стратегию в задачах поиска и преследования. В качестве иллюстрации изучим ситуацию: в условиях тумана катер береговой охраны преследует лодку браконьеров. В некоторый момент туман рассеивается, и лодка фиксируется на расстоянии k км от катера. Затем видимость снова исчезает, лодка продолжает движение прямолинейно, но направление неизвестно. Известно, что скорость катера в n раз больше скорости лодки.

Требуется определить, по какой траектории должен двигаться катер, чтобы гарантированно нагнать лодку.

2 Задание

1. Выполнить рассуждения и получить дифференциальные уравнения при условии, что скорость катера превосходит скорость лодки в n раз.
2. Построить траектории катера и лодки для двух вариантов начальных условий.
3. По графику определить точку встречи (пересечения траекторий).

3 Выполнение лабораторной работы

Положим $t_0 = 0$. Зафиксируем точку обнаружения лодки как начало отсчёта: $X_{\text{boat}}(0) = 0$. В момент обнаружения катер находится на расстоянии k , то есть относительно лодки $X_{\text{cutter}}(0) = k$ (с учётом выбора направления вдоль линии визирования).

Перейдём к полярным координатам. Полюс — точка обнаружения лодки: $x_0 = 0$ (при этом $\theta = x_0 = 0$), а полярная ось r направлена через точку положения катера береговой охраны.

3.1 Нахождение радиуса перехода x

Пусть через время t катер и лодка окажутся на одинаковом расстоянии x от полюса. За это время лодка пройдёт путь x (со скоростью v), а катер — $x + k$ или $x - k$ (в зависимости от того, по какую сторону от полюса он расположен относительно выбранного направления).

Тогда время движения для лодки равно x/v , а для катера — $(x + k)/(nv)$ или $(x - k)/(nv)$, поскольку скорость катера равна nv . Так как момент достижения радиуса x общий, получаем равенства:

- для первого случая: $x/v = (x + k)/(nv)$;
- для второго случая: $x/v = (x - k)/(nv)$.

Из этих соотношений получаем два характерных расстояния:

- $x_1 = \frac{k}{n+1}$ при $\theta = 0$;
- $x_2 = \frac{k}{n-1}$ при $\theta = -\pi$.

Далее задачу исследуем для обоих вариантов начального положения.

3.2 Разложение скорости катера и система ОДУ

После того как катер окажется на том же радиусе, что и лодка, прямолинейное движение уже не гарантирует перехват при неизвестном направлении. Поэтому катер должен начать движение вокруг полюса, одновременно удаляясь от него с радиальной скоростью, равной скорости лодки v .

Разложим скорость катера на компоненты:

- радиальная: $v_r = \frac{dr}{dt}$;
- тангенциальная: $v_t = r \frac{d\theta}{dt}$.

Чтобы катер не уступал лодке по удалению от полюса, задаём

$$(dr/dt) = v.$$

Полная скорость катера равна nv , значит по теореме Пифагора для ортогональных компонент:

$$v_t = \sqrt{(nv)^2 - v^2} = v\sqrt{n^2 - 1}.$$

Отсюда:

$$r \frac{d\theta}{dt} = v\sqrt{n^2 - 1}.$$

Итак, исходная задача сводится к решению системы:

$$\begin{cases} \frac{dr}{dt} = v, \\ r \frac{d\theta}{dt} = v\sqrt{n^2 - 1}. \end{cases}$$

3.2.1 Начальные условия

Случай 1

$$\begin{cases} \theta_0 = 0, \\ r_0 = \frac{k}{n+1}. \end{cases}$$

Случай 2

$$\begin{cases} \theta_0 = -\pi, \\ r_0 = \frac{k}{n-1}. \end{cases}$$

Исключая t из системы, получаем уравнение траектории:

$$\frac{dr}{d\theta} = \frac{r}{\sqrt{n^2 - 1}}.$$

Начальные условия остаются прежними. Решение этого ОДУ задаёт искомую траекторию катера в полярных координатах. После получения аналитической формы и численного решения можно построить траектории катера и лодки для обоих случаев.

3.3 Условие задачи

В тумане катер береговой охраны преследует лодку браконьеров. В момент, когда видимость временно улучшается, лодка обнаруживается на расстоянии 20 км от катера. Затем лодка снова скрывается и движется прямолинейно в неизвестном направлении. Скорость катера в 5 раз больше скорости лодки, то есть $k = 20$, $n = 5$.

Для численного моделирования и построения графиков использовались внешние файлы с программным кодом:

4 Полярные траектории: катер (ODE) и лодка (аналитически)

Цель: построить траектории в полярных координатах для «катера» (решение ОДУ) и «лодки» (заданная прямая в декартовых координатах, переведённая в полярные).

4.1 Инициализация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using Plots
using DataFrames
using JLD2

script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE))
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

4.2 Параметры задачи

```
n = 5
s = 20
fi = 3/4 * pi

p = (n = n, s = s, fi = fi)
```

4.3 Определение модели

Катер: $dr/d\theta = r / \sqrt{n^2 - 1}$ Здесь независимая переменная — θ (мы используем t как θ , как принято в ODEProblem).

```
function cutter_ode!(dr, r, p, □)
    dr[1] = r[1] / sqrt(p.n^2 - 1)
end
```

Лодка: линия $x(t)=t$, $y(t)=\tan(fi+\pi)*t$, затем перевод в (r, θ) В Julia правильнее использовать $\text{atan}(y, x)$, чтобы угол был в верном квадранте.

```
function boat_polar(t, p)
    k = tan(p.fi + pi)
    x = t
    y = k * t
    r = hypot(x, y)          # sqrt(x^2 + y^2)
    □ = atan(y, x)
    return r, □
end
```

Утилита: генерация траектории лодки для набора t

```

function make_boat_curve(tgrid, p)
    r = VectorFloat64(undef, length(tgrid))
    ϕ = VectorFloat64(undef, length(tgrid))
    for (i, tt) in pairs(tgrid)
        ri, ϕi = boat_polar(tt, p)
        r[i] = ri
        ϕ[i] = ϕi
    end
    return r, ϕ
end

```

Утилита: один прогон (и график, и таблицы, и сохранение)

```

function run_case(case_name; r0, ϕspan=(0.0, 2pi), nϕ=10_000, tmin=1e-9, tmax=8.0, nt

```

— Катер (ODE) —

```

ϕgrid = collect(LinRange(ϕspan[1], ϕspan[2], nϕ))
prob = ODEProblem(cutter_ode!, [r0], ϕspan, p)
sol = solve(prob, Tsit5(), saveat=ϕgrid)

df_cutter = DataFrame(ϕ = sol.t, r = first.(sol.u))

```

— Лодка (аналитически) —

```

tgrid = collect(LinRange(tmin, tmax, nt))
r_boat, ϕ_boat = make_boat_curve(tgrid, p)
df_boat = DataFrame(t = tgrid, ϕ = ϕ_boat, r = r_boat)

```

— Визуализация —

```
plt = plot(sol, proj=:polar, label="катер", xlabel="", ylabel="r",
           title="Полярные траектории – $case_name", lw=2, legend=:topleft)
plot!(plt, r_boat, proj=:polar, label="лодка", lw=2)
```

— Сохранение —

```
savefig(plt, plotsdir(script_name, "polar_$case_name.png"))
@save datadir(script_name, "data_$case_name.jld2") df_cutter df_boat p r0 rspan n

return (plt=plt, df_cutter=df_cutter, df_boat=df_boat)
end
```

4.4 Запуск 1: $r_0 = s/(n+1)$, $t \in (1e-9, 8)$

```
case1_r0 = p.s / (p.n + 1)
res1 = run_case("r0=s_div_(n+1)"; r0=case1_r0, tmax=8.0, p=p)

println("Кейс 1 – первые 5 строк (катер):")
println(first(res1.df_cutter, 5))
println("\nКейс 1 – первые 5 строк (лодка):")
println(first(res1.df_boat, 5))
```

4.5 Запуск 2: $r_0 = s/(n-1)$, $t \in (1e-9, 15)$

```
case2_r0 = p.s / (p.n - 1)
res2 = run_case("r0=s_div_(n-1)"; r0=case2_r0, tmax=15.0, p=p)
```

```
println("\nКейс 2 – первые 5 строк (катер):")
println(first(res2.df_cutter, 5))
println("\nКейс 2 – первые 5 строк (лодка):")
println(first(res2.df_boat, 5))
```

(опционально) показать последний график в интерактивной среде

```
res2.plt
```


5 Параметрическое исследование: катер (ODE) и лодка (аналитически) в полярных координатах

5.1 Активация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools
```

Установка каталогов

```
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

5.2 Определение модели

Катер: $dr/d\theta = r / \sqrt{n^2 - 1}$

```
function cutter_ode!(dr, r, p, θ)
    dr[1] = r[1] / sqrt(p.n^2 - 1)
end
```

Лодка: $x=t, y=\tan(\theta+\pi)*t \rightarrow (r, \theta)$ Важно: используем $\text{atan}(y, x)$, чтобы угол был корректен по квадрантам

```
function boat_polar(t, p)
    k = tan(p.fi + pi)
    x = t
    y = k * t
    r = hypot(x, y)
    θ = atan(y, x)
    return r, θ
end
```

5.3 Определение параметров в Dict

Все параметры — в одном Dict, как в шаблоне. Здесь «case» управляет выбором r_0 : $:plus(s/(n+1))$ или $:minus(s/(n-1))$.

```
base_params = Dict(
    :n => 5,
    :s => 20,
    :fi => 3/4*pi,
```

```

:case => :plus,                # :plus или :minus
:span => (0.0, 2*pi),          # интервал по  $\phi$ 
:nphi => 10_000,               # число точек для сохранения решения катера

:tspan_boat => (1e-9, 8.0),    # интервал по  $t$  для лодки
:nt_boat => 1_000,             # число точек для лодки

:solver => Tsit5(),
:experiment_name => "base_experiment"
)

println("Базовые параметры эксперимента:")
for (key, value) in base_params
    println(" $key = $value")
end

```

5.4 Функция-обертка для запуска одного эксперимента

Возвращаем Dict со строковыми ключами (как в твоём шаблоне).

```

function run_single_experiment(params::Dict)
    @unpack n, s, fi, case, span, nphi, tspan_boat, nt_boat, solver = params

```

Параметры для ODE в виде именованного кортежа

```
p = (n=n, s=s, fi=fi)
```

Начальное условие r_0 зависит от кейса

```
r0 = case == :plus ? s/(n+1) : s/(n-1)
```

— Катер (ODE) —

```
θgrid = collect(LinRange(θspan[1], θspan[2], nθ))  
prob = ODEProblem(cutter_ode!, [r0], θspan, p)  
sol = solve(prob, solver; saveat=θgrid)  
  
r_cutter = first.(sol.u)
```

— Лодка (аналитика) —

```
tgrid = collect(LinRange(tspan_boat[1], tspan_boat[2], nt_boat))  
r_boat = Vector{Float64}(undef, length(tgrid))  
θ_boat = Vector{Float64}(undef, length(tgrid))  
for (i, tt) in pairs(tgrid)  
    ri, θi = boat_polar(tt, p)  
    r_boat[i] = ri  
    θ_boat[i] = θi  
end
```

— Мини-анализ (несколько метрик, чтобы было что сравнивать в сканировании) — 1) финальный радиус катера на $\theta=\text{end}$

```
r_cutter_final = r_cutter[end]
```

2) максимальный радиус лодки на её сетке

```
r_boat_max = maximum(r_boat)
```

3) простая «разница масштабов» (без строгого физического смысла, но для сравнения кейсов полезно)

```

scale_ratio = r_cutter_final / r_boat_max

return Dict(
    "solution" => sol,
    "□_points" => sol.t,
    "r_cutter" => r_cutter,

    "t_points_boat" => tgrid,
    "□_boat" => □_boat,
    "r_boat" => r_boat,

    "r0" => r0,
    "r_cutter_final" => r_cutter_final,
    "r_boat_max" => r_boat_max,
    "scale_ratio" => scale_ratio,

    "parameters" => params
)
end

```

5.5 Запуск базового эксперимента (кэширование)

```

data_base, path_base = produce_or_load(
    datadir(script_name, "single"),
    base_params,
    run_single_experiment;
    prefix = "polar",

```

```

    tag = false,
    verbose = true
)

println("\nРезультаты базового эксперимента:")
println(" r0: ", data_base["r0"])
println(" r_cutter_final: ", data_base["r_cutter_final"])
println(" r_boat_max: ", data_base["r_boat_max"])
println(" scale_ratio: ", round(data_base["scale_ratio"]; digits=4))
println(" Файл результатов: ", path_base)

```

5.6 Визуализация базового эксперимента

```

p1 = plot(
    data_base["□_points"], data_base["r_cutter"],
    proj=:polar,
    label="катер",
    xlabel="□",
    ylabel="r",
    title="Базовый эксперимент (case=$(base_params[:case]))",
    lw=2,
    legend=:topleft,
    grid=true
)
plot!(
    p1,
    data_base["□_boat"], data_base["r_boat"],

```

```

    proj=:polar,
    label="лодка",
    lw=2
)

savefig(p1, plotsdir(script_name, "single_experiment.png"))

```

5.7 Второй базовый эксперимент (как у тебя во 2-й части): case=:minus, tmax=15

```

base_params2 = copy(base_params)
base_params2[:case] = :minus
base_params2[:tspan_boat] = (1e-9, 15.0)
base_params2[:experiment_name] = "base_experiment_minus"

data_base2, path_base2 = produce_or_load(
    datadir(script_name, "single"),
    base_params2,
    run_single_experiment;
    prefix = "polar",
    tag = false,
    verbose = true
)

p1b = plot(
    data_base2["□_points"], data_base2["r_cutter"],
    proj=:polar,

```

```

    label="катер",
    xlabel="",
    ylabel="r",
    title="Базовый эксперимент (case=$(base_params2[:case]))",
    lw=2,
    legend=:topleft,
    grid=true
)
plot!(
    p1b,
    data_base2["□_boat"], data_base2["r_boat"],
    proj=:polar,
    label="лодка",
    lw=2
)
savefig(p1b, plotsdir(script_name, "single_experiment_minus.png"))

```

5.8 Параметрическое сканирование

В твоей задаче естественно сканировать n (оно влияет и на ODE, и на r_0). При желании можно заменить на $:fi$ или $:s$ (или сделать сетку по двум параметрам).

```

param_grid = Dict(
    :n => [3, 4, 5, 6, 8, 10],      # сканируем n
    :s => [20],                    # фиксируем
    :fi => [3/4*pi],               # фиксируем

    :case => [:plus, :minus],      # сканируем оба кейса

```



```

:span => [(0.0, 2*pi)],
:n => [10_000],

:tspan_boat => [(1e-9, 8.0)],    # можно тоже сканировать, но обычно фиксируют
:nt_boat => [1_000],

:solver => [Tsit5()],
:experiment_name => ["parametric_scan"]
)

all_params = dict_list(param_grid)

println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("Всего комбинаций параметров: ", length(all_params))
println("Исследуемые n: ", param_grid[:n])
println("Исследуемые case: ", param_grid[:case])
println("="^60)

```

5.9 Запуск всех экспериментов и сбор результатов

```

all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
    println("Порядок: $i/$(length(all_params)) | n=$(params[:n]) | case=$(params[:case])")
    # ... (code for running experiments) ...
end

```

```

data, path = produce_or_load(
    datadir(script_name, "parametric_scan"),
    params,
    run_single_experiment;
    prefix = "scan",
    tag = false,
    verbose = false
)

```

Сводка по эксперименту

```

result_summary = merge(
    params,
    Dict(
        :r0 => data["r0"],
        :r_cutter_final => data["r_cutter_final"],
        :r_boat_max => data["r_boat_max"],
        :scale_ratio => data["scale_ratio"],
        :filepath => path
    )
)
push!(all_results, result_summary)

```

Полные данные (катер) — удобно для дальнейших графиков/анализа

```

df = DataFrame(
    □ = data["□_points"],
    r = data["r_cutter"],
    n = fill(params[:n], length(data["□_points"])),
    case = fill(string(params[:case]), length(data["□_points"]))
)

```

```

    )
    push!(all_dfs, df)
end

```

5.10 Анализ и визуализация результатов сканирования

```

results_df = DataFrame(all_results)
println("\nСводная таблица результатов (первые строки):")
println(first(results_df, 10))

```

Сравнительный график траекторий катера для всех комбинаций (по θ)

```

p2 = plot(size=(900, 520), dpi=150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    plot!(
        p2,
        data["□_points"], data["r_cutter"],
        label="n=$(params[:n]), case=$(params[:case])",

```

```

        lw=2,
        alpha=0.8
    )
end
plot!(
    p2,
    xlabel="□",
    ylabel="r(□)",
    title="Сканирование: траектории катера (ODE) при разных n и case",
    legend=:outerright,
    grid=true
)
savefig(p2, plotsdir(script_name, "parametric_scan_cutter_comparison.png"))

```

График метрики `scale_ratio` по `n` (раздельно для `case`) (`scale_ratio = r_cutter_final / r_boat_max`)

```

p3 = plot(size=(900, 520), dpi=150)
for cs in unique(results_df.case)
    sub = results_df[results_df.case .== cs, :]
    plot!(
        p3,
        sub.n, sub.scale_ratio,
        seriestype=:scatter,
        label="case=$cs"
    )
end
plot!(
    p3,

```

```

    xlabel="n",
    ylabel="scale_ratio",
    title="Зависимость scale_ratio от n (для разных case)",
    legend=:topleft,
    grid=true
)
savefig(p3, plotsdir(script_name, "scale_ratio_vs_n.png"))

```

5.11 Бенчмаркинг с разными параметрами

```

println("\n" * "="^60)
println("Бенчмаркинг для разных n (оба case)")
println("="^60)

benchmark_results = []

```

Возьмём сетку n из param_grid и оба case (как в сканировании)

```

for n_value in param_grid[:n], case_value in param_grid[:case]
    bench_params = Dict(
        :n => n_value,
        :s => base_params[:s],
        :fi => base_params[:fi],
        :case => case_value,
        :span => base_params[:span],
        :n[] => base_params[:n[]],
        :tspan_boat => base_params[:tspan_boat],
        :nt_boat => base_params[:nt_boat],
    )

```

```

        :solver => base_params[:solver]
    )

```

```

function benchmark_run()

```

Только катер (ODE) — это и есть «вычислительная часть»

```

    p = (n=bench_params[:n], s=bench_params[:s], fi=bench_params[:fi])
    r0 = bench_params[:case] == :plus ? bench_params[:s]/(bench_params[:n]+1) : b
    prob = ODEProblem(cutter_ode!, [r0], bench_params[:span], p)
    return solve(prob, bench_params[:solver]; saveat=LinRange(bench_params[:span]
end

    println("\nБенчмарк для n = $n_value, case = $case_value:")
    b = @benchmark $benchmark_run() samples=80 evals=1
    tsec = median(b).time / 1e9
    println(" Медианное время: ", round(tsec; digits=4), " сек")

    push!(benchmark_results, (n=n_value, case=string(case_value), time=tsec))
end

bench_df = DataFrame(benchmark_results)

```

График времени вычисления от n (раздельно по case)

```

p4 = plot(size=(900, 520), dpi=150)
for cs in unique(bench_df.case)
    sub = bench_df[bench_df.case .== cs, :]
    plot!(
        p4,

```

```

        sub.n, sub.time,
        seriestype=:scatter,
        label="case=$cs"
    )
end
plot!(
    p4,
    xlabel="n",
    ylabel="Время вычисления, сек",
    title="Зависимость времени решения ODE от n (для разных case)",
    legend=:topleft,
    grid=true
)
savefig(p4, plotsdir(script_name, "computation_time_vs_n.png"))

```

5.12 Сохранение всех результатов

```

@save datadir(script_name, "all_results.jld2") base_params base_params2 param_grid al
@save datadir(script_name, "all_plots.jld2") p1 p1b p2 p3 p4

println("\n" * "="^60)
println("ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА")
println("="^60)
println("\nРезультаты сохранены в:")
println(" • data/$(script_name)/single/ - базовые эксперименты")
println(" • data/$(script_name)/parametric_scan/ - параметрическое сканирование")
println(" • data/$(script_name)/all_results.jld2 - сводные данные")

```

```
println(" • plots/$(script_name)/ - все графики")
println(" • data/$(script_name)/all_plots.jld2 - объекты графиков")
println("\nДля анализа результатов используйте:")
println(" using JLD2, DataFrames")
println(" @load \"data/$(script_name)/all_results.jld2\"")
println(" println(results_df)")
```

5.13 Анализ результатов моделирования

В рамках работы выполнено численное исследование движения катера, задаваемого уравнением

$$\frac{dr}{d\theta} = \frac{r}{\sqrt{n^2 - 1}},$$

а также сопоставление полученной траектории со движением лодки, которое задано аналитически (как прямолинейное в декартовой системе) и далее представлено в полярных координатах.

5.14 Базовые эксперименты

5.14.1 1. Случай (case = plus)



Рисунок 5.1: Базовый эксперимент (case=plus)

На полярной диаграмме траектория катера имеет форму расходящейся спирали. Радиус r возрастает при увеличении угла θ , причём скорость роста увеличивается по мере удаления от центра. Это соответствует структуре уравнения: $dr/d\theta$ пропорциональна r , значит зависимость носит экспоненциальный характер по углу.

Лодка при этом движется по лучу: в декартовой системе это прямая, а в полярных координатах — траектория, соответствующая движению в фиксированном направлении.

Визуально заметно, что катер увеличивает радиус быстрее, и расстояние между кривыми меняется по мере движения.

5.14.2 2. Случай (case = minus)



Рисунок 5.2: Базовый эксперимент (case=minus)

Во втором варианте начальное значение радиуса больше, поэтому стартовая точка катера расположена дальше от полюса. Спираль сохраняет прежний тип роста, но вся траектория оказывается «сдвинутой наружу» по масштабу.

Таким образом, отличие режимов case=plus и case=minus связано главным образом с начальными условиями, а не с качественной формой траектории.

5.15 Параметрическое сканирование по n

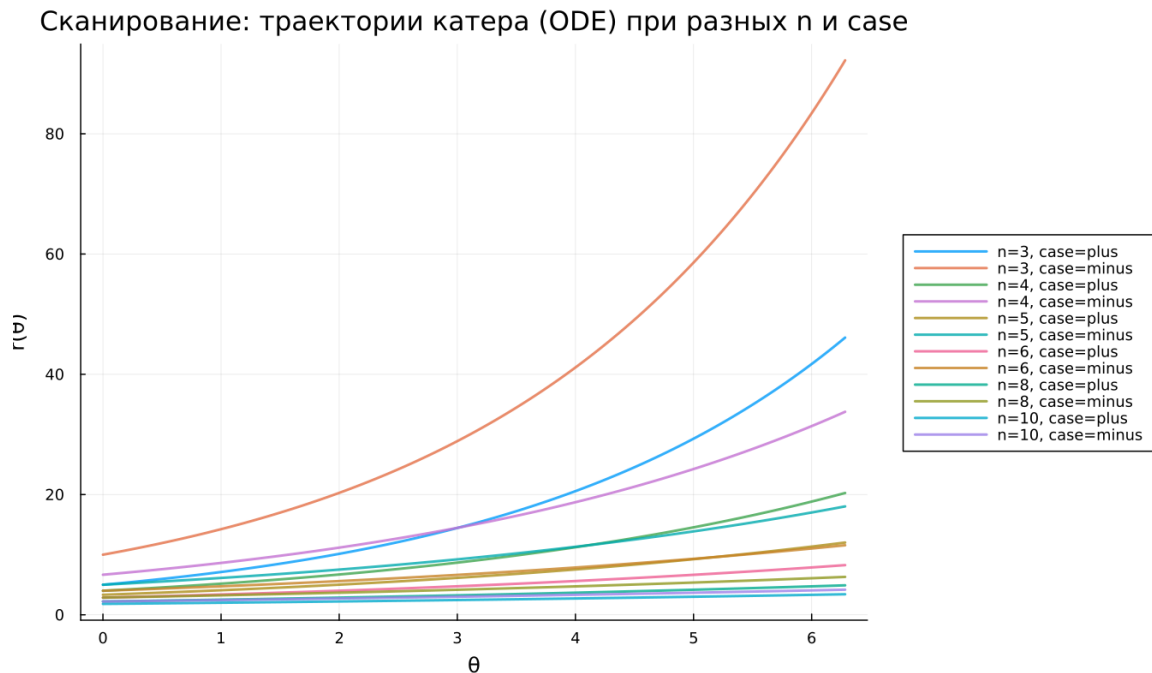


Рисунок 5.3: Сканирование траекторий катера

Исследовано влияние параметра n на динамику для обоих режимов начальных условий.

Из уравнения видно, что темп изменения задаётся коэффициентом $1/\sqrt{n^2 - 1}$. При росте n этот множитель уменьшается, поэтому:

- при меньших n спираль расходится заметно быстрее;
- при больших n радиальный рост становится более медленным;
- траектории выглядят более «пологими» и менее быстро уходят наружу.

На графике видно, что для $n = 3$ рост наиболее интенсивный, а для $n = 10$ — минимальный среди рассмотренных.

5.16 Анализ метрики scale_ratio

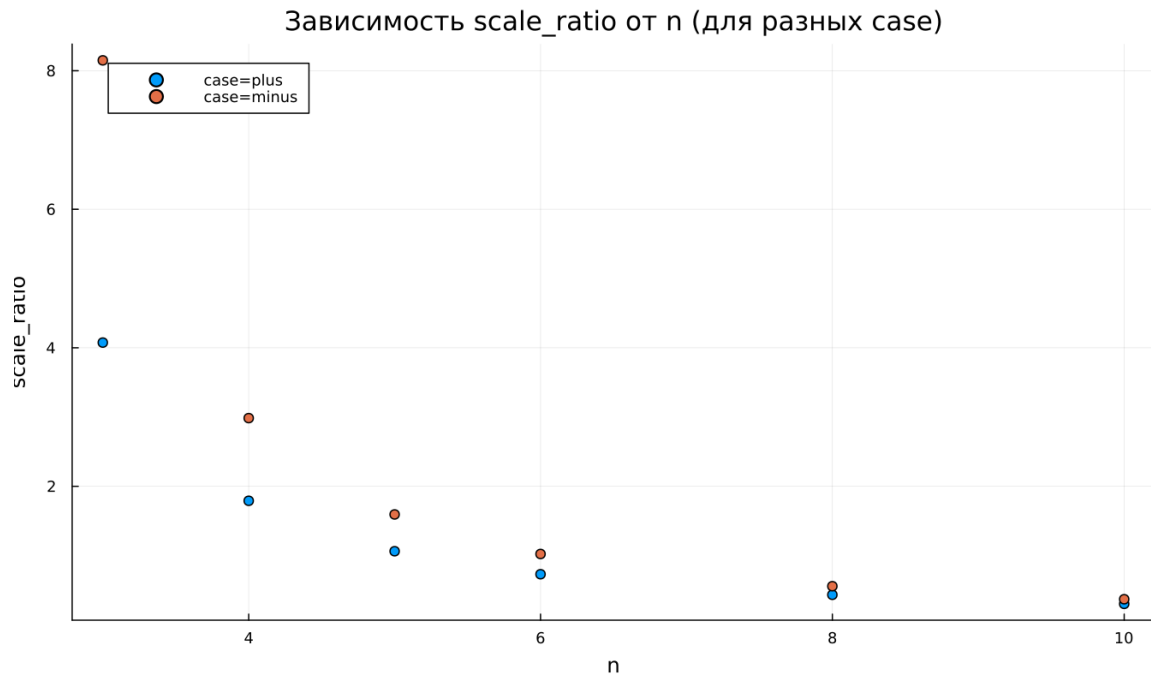


Рисунок 5.4: Зависимость scale_ratio от n

Введена метрика

$$\text{scale_ratio} = \frac{r_{\text{final}}}{\max(r_{\text{boat}})},$$

которая показывает относительный масштаб траектории катера по сравнению с максимальным радиусом лодки в рамках выбранного интервала моделирования.

По графику можно заключить:

- при малых n метрика существенно превышает 1 — катер «наращивает радиус» значительно быстрее лодки;
- при увеличении n значение заметно снижается;
- при больших n масштабы траекторий становятся ближе друг к другу.

Для режима case=minus значения выше, что объясняется большим стартовым радиусом.

5.17 Время вычислений

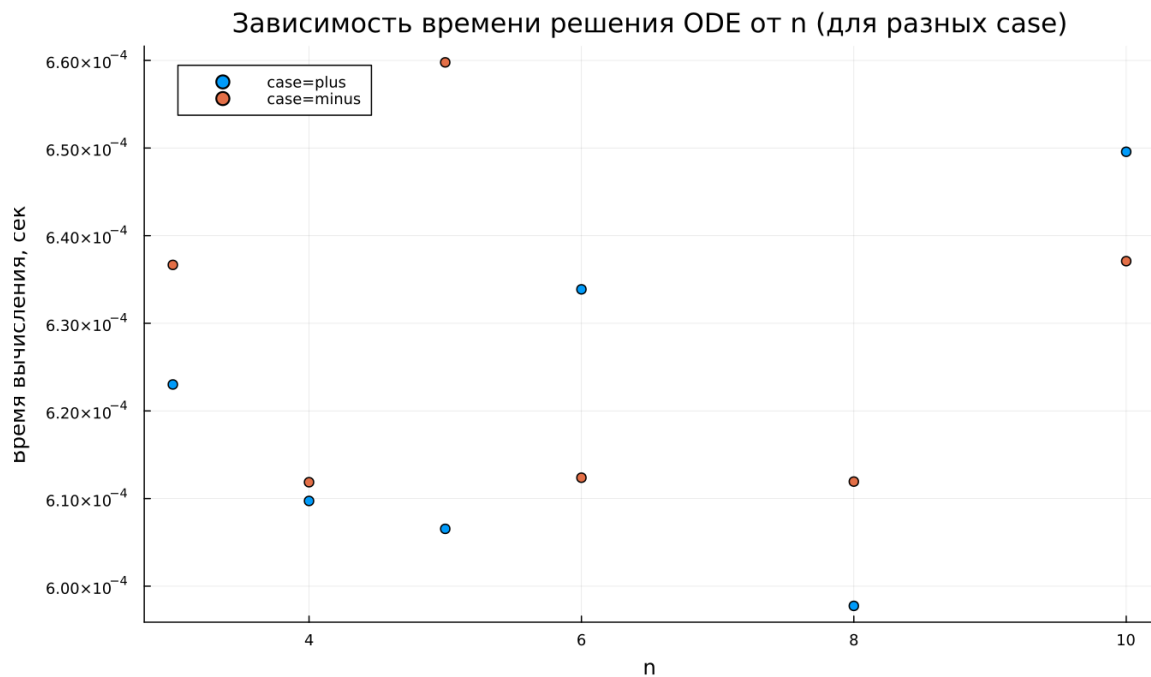


Рисунок 5.5: Зависимость времени вычисления от n

Проведён бенчмарк численного решения ОДУ при различных значениях n .

Наблюдения:

- время расчёта находится на уровне порядка 6×10^{-4} секунды;
- выраженной зависимости от n не выявлено;
- небольшие флуктуации обусловлены адаптивным выбором шага интегрирования и особенностями численных процедур.

6 Выводы

1. Траектория катера в полярных координатах описывается экспоненциально расходящейся спиралью.
2. Параметр n определяет «крутизну» спирали: чем больше n , тем медленнее растёт радиус r как функция θ .
3. Выбор начального условия (case) влияет на масштаб и стартовую позицию, но не меняет тип траектории.
4. Численная схема демонстрирует устойчивость, а затраты по времени слабо зависят от параметров модели.

Численные результаты согласуются с аналитическими выводами, полученными из структуры дифференциального уравнения.

Список литературы

1. Задача о погоне